DESIGN QUESTIONS

Design a Payment Gateway

Transaction Flow, Idempotency, PCI Compliance & Fraud Detection

~ 35 minute read

© Study Guide — For personal use only

What's Inside

01 · Problem Overview
02 · Clarifying Questions
03 · Capacity Est.
04 · Architecture
05 · Design Decisions
06 · Data Models
07 · Payment Flow
08 · Fraud Detection
09 · Reconciliation
10 · Failure Modes
11 · Interview Playbook

01 Problem Overview

Stripe/PayPal at Scale

Scale & Requirements

- **Volume:** 250M transactions/day; 2.9K TPS avg; 10K TPS peak
- **Uptime:** 99.999% (5 nines); money cannot be lost or duplicated
- **Latency:** <500ms authorization; <2s end-to-end charge
- **Compliance:** PCI DSS Level 1; never store raw card data; audit everything
- **Fraud:** Real-time ML scoring; 3DS authentication; velocity checks
- **Settlement:** Daily batch reconciliation with card networks; double-entry accounting

Core Challenges

- **Idempotency:** Networks timeout; must retry without duplicating charges
- **Double-spend:** User clicks 'Pay' twice; prevent before hitting bank
- **Distributed transactions:** Atomically update merchant, bank, fraud logs across regions
- **External dependency:** Bank takes 3–5s; must have async fallback
- **Fraud arms race:** ML models retrain daily; balance false positives vs. negatives
- **Compliance:** PCI DSS, PSD2, local regs; breach = $5M–$25M fines

02 Clarifying Questions

Scope Refinement

Question	Options	Assumption
Payment types?	Cards, ACH, wallets, crypto	Credit/debit cards
Multi-currency?	Single (USD) or global	Multi-currency (100+)
Regions?	US-only or global	Global (different regs)
Merchants?	Small/large/platforms	Mix of all types
PCI Level?	1, 2, or 3	Level 1 (strictest)
Retry logic?	Auto-retry? Fallback?	3 retries; cached fallback <30s
Refunds?	Full/partial? Chargebacks?	Full + partial; chargeback flow

© Study Guide — For personal use only

03 Capacity Estimation

Quantify the Scale

Storage & Latency

- **Payments DB:** 250M × 365 × 5yr = 450TB hot + 2.3PB archive; 5KB per record
- **Ledger:** 3–4 entries/charge; 1B/day = 5TB hot (year) + 50TB archive
- **Idempotency cache (Redis):** 250M keys × 24h TTL × 10KB = 2.5TB
- **Latency targets:** fraud check <50ms; db write <20ms; end-to-end <2s (includes 3–5s bank latency)

© Study Guide — For personal use only

04 High-Level Architecture

System Components & Data Flow

© Study Guide — For personal use only

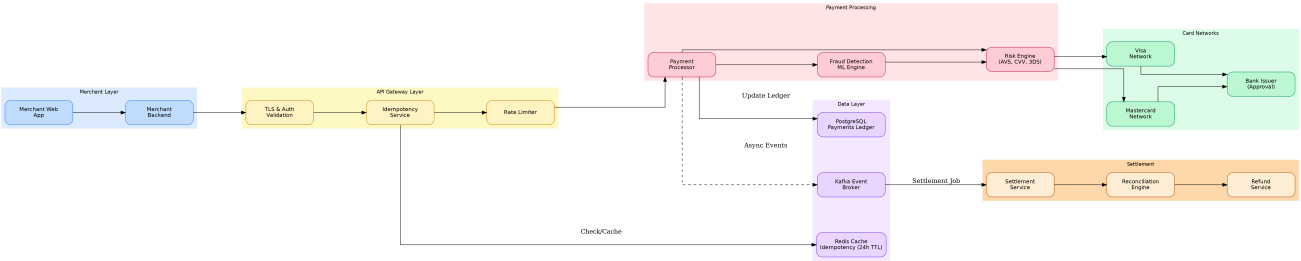

Figure 1: *High-Level Architecture. Merchants submit charges via REST API to the API Gateway (TLS, idempotency, rate limiting). The Payment Processor orchestrates fraud detection, risk checks, and card network authorization. Data flows through PostgreSQL (ledger), Redis (idempotency cache, 24h TTL), and Kafka (async events). Separate settlement and refund services handle batch reconciliation and reversals.*

 © Study Guide — For personal use only

System Architecture

Tier 1: API Gateway

- **Clients:** REST API, mobile SDKs, webhooks
- **Rate limiting:** Token bucket per merchant key
- **Auth:** TLS 1.3 + HMAC-SHA256 signature verification
- **Load balancing:** Route53 latency-based routing; multi-region

Tier 2: Payment Services

- **Processor:** Orchestrates charge flow; calls fraud/risk/card network
- **Fraud:** ML model (LightGBM); real-time scoring <50ms; score 0–100
- **Risk:** AVS, CVV, 3DS, velocity checks
- **Reconciliation:** Daily batch matching; discrepancy alerts

Tier 3: Data Layer

- **PostgreSQL:** ACID; WAL replication; multi-region master-slave
- **Ledger:** Double-entry accounting; immutable append-only; 3–4 entries per charge
- **Redis:** → mappings; 24h TTL; cluster HA
- **Kafka:** Async events (payment.authorized, .declined, .refunded); consumers: webhooks, logs, analytics

Tier 4: External & Compliance

- **Card Networks:** Visa, Mastercard, Amex; ISO 8583 + REST APIs
- **Acquirers:** Banks processing payments on merchant's behalf
- **Issuing Banks:** Consumer banks; approve/decline based on funds, fraud rules
- **PCI DSS Vault:** HSM; tokenize raw cards at entry point; never in app code
- **Audit Logs:** Immutable append-only; every transaction, every action; PCI requirement

05 Core Design Decisions

Idempotency, At-Least-Once, Double-Spend

Decision 1: Idempotency Keys

- **Problem:** Merchant clicks 'Pay'; network timeout; merchant retries; duplicate charge?
- **Solution:** Client provides idempotency_key (UUID). Server stores key → response in Redis for 24h.
- **Mechanism:** First request: process → store . Retry: return cached response immediately.
- **Guarantee:** Exactly-once semantics at API layer; no double charges

© Study Guide — For personal use only

Decision 2: At-Least-Once + Idempotency (Kafka)

- **Problem:** Payment completes; webhook to merchant fails; merchant never learns
- **Solution:** Publish event to Kafka; retry queue exponential backoff (1s, 10s, 100s, 1000s); max 5 retries
- **Guarantee:** At-least-once delivery; merchant may see duplicates; must check event ID

Decision 3: Double-Spend Prevention

- **Race condition:** Both requests reach server simultaneously
- **Solution:** Idempotency key deduplicated; advisory lock in PostgreSQL prevents concurrent processing
- **Fallback:** Optimistic locking; flag charges <1s apart with overlapping amounts as suspicious

06 Data Models

Schema & Storage

Payments Table (PostgreSQL)

```
CREATE TABLE payments (
id UUID PRIMARY KEY,
merchant_id UUID NOT NULL,
idempotency_key UUID UNIQUE,
amount_cents BIGINT NOT NULL,
currency CHAR(3), -- USD, EUR, GBP
status ENUM('pending','authorized','captured','declined'),
card_token VARCHAR(255), -- Tokenized, never raw
fraud_score DECIMAL(5,2), -- 0–100; >80 = manual review
auth_code VARCHAR(10), -- From card network
created_at TIMESTAMPTZ,
captured_at TIMESTAMPTZ,
settled_at TIMESTAMPTZ,
INDEX (merchant_id, created_at),
INDEX (status, created_at)
);
```

ACID transactions; WAL durability; searchable

Ledger Entries (Double-Entry Accounting)

```
CREATE TABLE ledger_entries (
```


© Study Guide — For personal use only

```
id UUID PRIMARY KEY,
transaction_id UUID, -- Groups 3–4 entries
account_id UUID,
debit_cents BIGINT, -- Money OUT
credit_cents BIGINT, -- Money IN
currency CHAR(3),
description VARCHAR(255), -- 'Charge','Fee',etc
created_at TIMESTAMPTZ,
settled_at TIMESTAMPTZ,
INDEX (account_id, created_at)
);
-- Invariant: sum(debits) = sum(credits) [verified nightly]
```

Immutable append-only accounting log

Idempotency Keys

```
-- Redis (fast):
Key: 'idempotency:{key}'
Value: {payment_id, status, amount, currency}
TTL: 86400 seconds
-- PostgreSQL (durable):
CREATE TABLE idempotency_keys (
key UUID PRIMARY KEY,
merchant_id UUID,
response_body JSONB,
created_at TIMESTAMPTZ,
expires_at TIMESTAMPTZ
);
```

Request deduplication & replay protection

© Study Guide — For personal use only

07 Payment Transaction Flow

Complete Lifecycle

© Study Guide — For personal use only

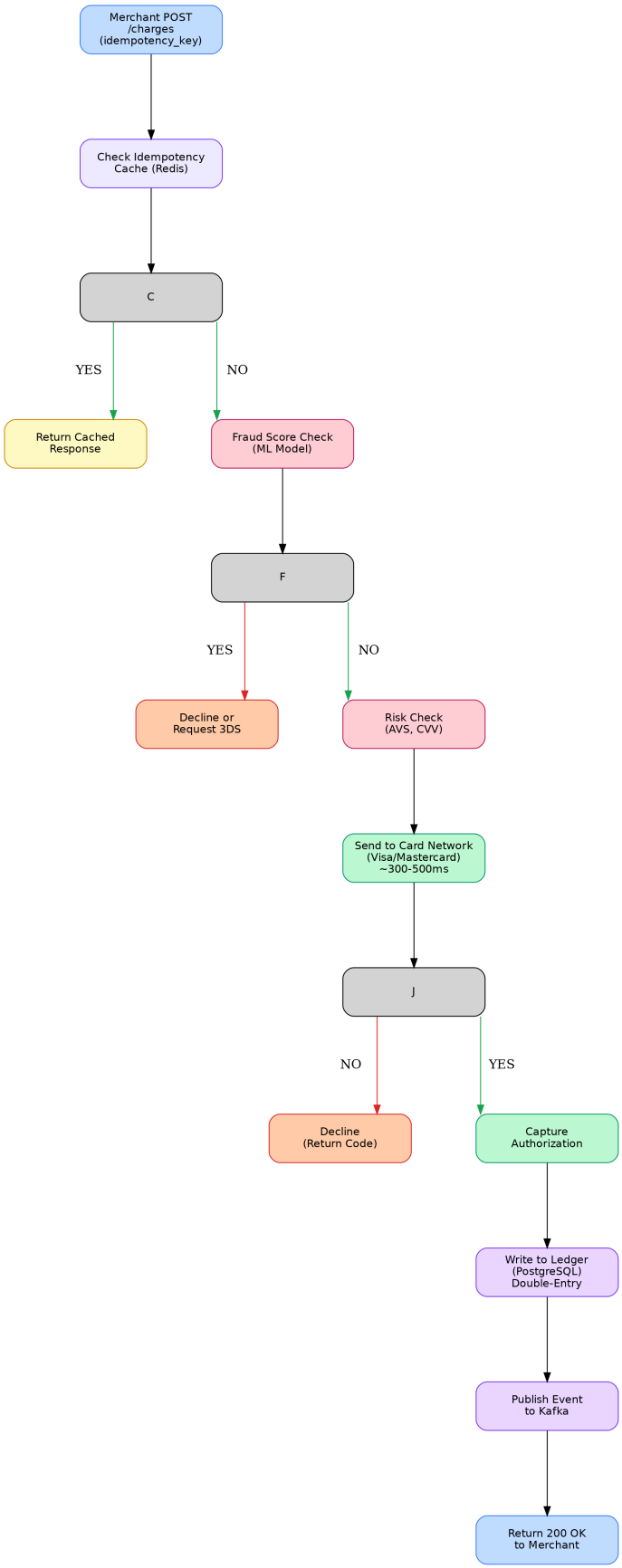

Figure 2: *Payment Transaction Flow. (1) Merchant POST /charges with idempotency_key. (2) Check Redis cache; if duplicate, return cached response immediately. (3) Run fraud detection (ML model); if score exceeds threshold, decline or prompt for 3DS. (4) Risk checks (AVS, CVV). (5) Send to card network (~300ms latency). (6) Await bank approval. (7)*

 © Study Guide — For personal use only

Capture authorization. (8) Update PostgreSQL ledger (double-entry). (9) Publish async event to Kafka. (10) Return 200 OK. Green arrows = happy path; red dashed = rejection paths.

 © Study Guide — For personal use only

08 Fraud Detection & Prevention

ML + Rules + 3DS

Real-Time ML Model

- **Algorithm:** LightGBM (gradient boosting); retrained daily on last 30 days
- **Features:** Merchant (MCC, chargeback %), card (age, country), user (device, IP, geolocation), transaction (amount, currency)
- **Output:** fraud_score 0–100; merchant sets threshold (70=decline, 85=review)
- **Latency:** <50ms; served via Seldon or KServe
- **Monitoring:** Track precision, recall, AUC-ROC daily; rebalance classes (fraud ~0.5%)

Deterministic Rules + 3DS

- **Velocity:** Same card 5x in 5 min → decline
- **Geo anomalies:** Card issued US, txn from Russia 1h ago, now UK → risky
- **3DS:** Cardholder SMS OTP or biometric; 30–120s latency; liability shift if authenticated
- **Device fingerprinting:** Client-side JS collects user agent, screen res, timezone, IP; detect stolen creds

© Study Guide — For personal use only

09 Settlement & Reconciliation

Daily Accounting Cycle

© Study Guide — For personal use only

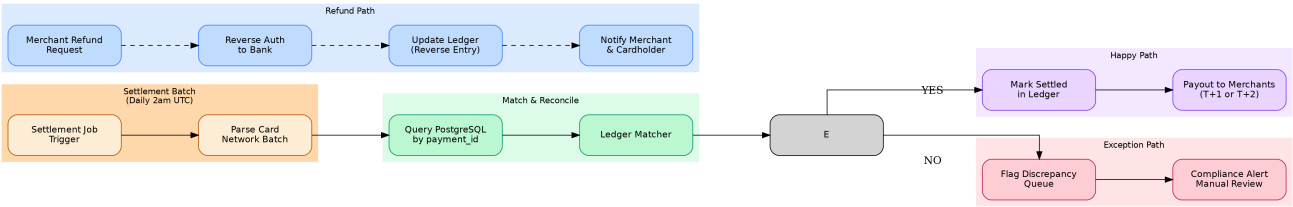

Figure 3: *Settlement & Reconciliation Flow. Daily batch job (2am UTC) receives settlement file from card networks. System matches payment_id and amount against PostgreSQL ledger. Matches are marked settled; discrepancies trigger compliance alerts and manual review. Refund path: merchant request → reverse authorization → update ledger → notify cardholder (3–5 business days). Double-entry invariant (debits = credits) verified nightly.*

 © Study Guide — For personal use only

10 Failure Modes & Recovery

8 Scenarios

Failure	Cause	Impact	Mitigation
Bank Timeout	Issuing bank API >5s	Auth hangs; retry; potential duplicate	Hard timeout 5s; soft decline; idempotency prevents duplicate
Network Partition	ISP/CDN routing down	Merchants can't reach API	Multi-region active-active; Route53 failover <30s
Duplicate Charge	Retry same idempotency key	Same charge twice	Redis/DB lookup; return cached response
Fraud False Positive	ML model too strict	Good customer declined; lost revenue	A/B test thresholds; monitor false positive rate
PCI DSS Breach	Database hacked; card data exposed	Regulatory fine $5M–$25M	Never store raw cards; tokenize; PCI audit; encryption
Card Network Outage	Visa/Mastercard APIs down (rare)	Cannot authorize any txns	Cached pre-approved list; local fallback rules
Redis Eviction	Memory full; old keys evicted prematurely	Old idempotency key expired; duplicate charge	2.5TB memory; monitor eviction rate

11 Interview Playbook

45-Minute Presentation

Timeline: 5+3+10+20+5 minutes

- **Opening (5 min):** Clarifying questions (payment types? scope? compliance?) → state assumptions
- **Problem (3 min):** 250M txn/day = 2.9K TPS; 5s card network latency; bottleneck = everything else must be <500ms
- **Architecture (10 min):** Show Diagram 1; explain 4 tiers; solid/dashed arrows for sync/async
- **Deep dives (20 min):** Pick 2–3: idempotency keys, payment flow (diagram 2), fraud ML, reconciliation (diagram 3), resilience
- **Wrap-up (5 min):** Trade-offs (at-least-once vs. exactly-once; batch vs. real-time settlement); complexity of compliance

© Study Guide — For personal use only

Key Numbers to Memorize

- 250M txn/day; 2.9K TPS avg; 10K TPS peak
- Card network latency: 3–5 seconds (out of control)
- API latency: <500ms; fraud check <50ms
- Uptime: 99.999% = 26 seconds downtime/year
- Idempotency cache: 2.5TB Redis (24h TTL)
- Ledger: 3–4 entries/charge; 1B/day
- Settlement: daily batch; 2–5 day latency

Common Probe Questions

- 'How do you prevent double-charges?' → Idempotency keys + deduplication
- 'Card network timeout — what do you do?' → Soft decline, retry, cached fallback
- 'Why double-entry accounting?' → Invariant checking (debits = credits); audit trail
- 'How does 3DS work?' → Cardholder auth; liability shift
- 'Multi-region or single?' → Active-active for 99.999% uptime

■ KEY INSIGHT

Payment systems demand obsessive correctness and resilience. Idempotency is critical. Test failure modes relentlessly: mock timeouts, simulate database failures, verify ledger invariants.

© Study Guide — For personal use only